WNDRVR
SUPPORT NETWORK

API REFERENCE: KERNEL OPEN SOURCE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

API Reference: Kernel Open Source, SR0650

20 July 2020

1. XMLCANON

Name

xmlCanon - XML parser canonicalization module

Layer

EXPAT

Routines

xmlCanonFile( ) - canonicalizes an XML document and places the output into the file specified by outputDocName
xmlCanonParamInit( ) - sets up configuration parameters for the canonicalizer currently this routine is unused

Description

This module provides the means to canonicalize an existing valid XML document. All output in this module is done by the XML output module, therefore it must be included to use this module.

see Canonical XML Version 1.0 specs at http://www.w3.org/TR/xml-c14n for canonicalization specifications. Below is an excerpt defining canonical form.

"The canonical form of an XML document is physical representation of the document produced by the method described in this specification. The changes are summarized in the following list:

- The document is encoded in UTF-8
- Line breaks normalized to #xA on input, before parsing
- Attribute values are normalized, as if by a validating processor
- Character and parsed entity references are replaced
- CDATA sections are replaced with their character content
- The XML declaration and document type declaration (DTD) are removed
- Empty elements are converted to start-end tag pairs
- Whitespace outside of the document element and within start and end tags is normalized
- All whitespace in character content is retained (excluding characters removed during line feed normalization)
- Attribute value delimiters are set to quotation marks (double quotes)
- Special characters in attribute values and character content are replaced by character references
- Superfluous namespace declarations are removed from each element
- Default attributes are added to each element

WNDRVR

- Lexicographic order is imposed on the namespace declarations and attributes of each element

The term canonical XML refers to XML that is in canonical form. The XML canonicalization method is the algorithm defined by this specification that generates the canonical form of a given XML document or document subset. The term XML canonicalization refers to the process of applying the XML canonicalization method to an XML document or document subset.

If an XML document must be converted to a node-set, XPath REQUIRES that an XML processor be used to create the nodes of its data model to fully represent the document. The XML processor performs the following tasks in order:

- normalize line feeds
- normalize attribute values
- replace CDATA sections with their character content
- resolve character and parsed entity references"

Includes: **stdio.h**, **stdlib.h**, **stddef.h**, **string.h**, **errnoLib.h**, **expat.h**, **codepage.h**, **xmlfile.h**, **xmltchar.h**, **xmlCanon.h**, **xmlop.h**

Include files

none

Name

xmlCanonFile( ) - canonicalizes an XML document and places the output into the file specified by outputDocName

Layer

EXPAT

Synopsis

```
void xmlCanonFile
    (
    const char *canonizeDocName,   /* name of document to canonicalize */
    const char *outputDocName      /* name of document to place output in */
    )
```

Description

This routine takes the XML document specified by canonizeDocName, canonicalizes it, and places the canonicalized file output into the file specified by outputDocName. Both document names must specify the full path to the document. Namespaces will be used; all the parameter entities are parsed. If xmlCanonFile fails the outputDoc will not exist, or will have a zero length.

Returns

N/A

Errno

N/A

See also

xmlCanon

Name

xmlCanonParamInit( ) - sets up configuration parameters for the canonicalizer currently this routine is unused

Layer

EXPAT

Synopsis

```
void xmlCanonParamInit(void)
```

Description

This routine is used to setup configuration parameters for the XML canonicalizer. Currently it is unused, but it is typically used by the Tornado project facility for component setup. The empty routine is included is to ensure that the project facility inlcudes the xmlparse object into the image. If there are no calls to the functions within the object, the object is optimized out automatically.

Returns

N/A

Errno

N/A

See also

xmlCanon

2. XMLDOC

Name

xmlparse - XML parser based on Expat 2.2.7

Layer

EXPAT

Routines

XML_ParserCreate( ) - creates a parser that is not concerned with namespaces or an external memory suite **XML_ParserCreateNS( )** - creates a parser that is not concerned with an external memory suite **XML_ParserCreate_MM( )** - general parser object setup and initialization **XML_ParserReset( )** - re-initializes the whole parser object **XML_SetEncoding( )** - sets the encoding for the XML parser, indicating how the document to be parsed is encoded **XML_ExternalEntityParserCreate( )** - creates an XML_Parser object that can parse an external general entity **XML_ParserFree( )** - frees all memory used by the parser object passed in **XML_UseParserAsHandlerArg( )** - sets the m_handlerArg member variable of the parser passed in to be the parser **XML_UseForeignDTD( )** - forces the parser to assume that there is an external subset **XML_SetReturnNSTriplet( )** - sets how namespace triplets are displayed, e.g. URI + seperator + local_name **XML_SetUserData( )** - assigns an arbitrary piece of data/structure to be passed throughout routines as userData **XML_SetBase( )** - sets the base directory of the parser object, for relative paths used in entity handling **XML_GetBase( )** - retrieves the base path stored in the parser object **XML_GetSpecifiedAttributeCount( )** - returns the number of the attribute/value pairs specified in the last element, not including default attributes **XML_GetIdAttributeIndex( )** - retrieves the ID attribute index passed in the last call to XML_StartElementHandler **XML_SetElementHandler( )** - assigns the start and end element callbacks to the passed in parser object **XML_SetStartElementHandler( )** - assigns the start element callback to the passed in parser object **XML_SetEndElementHandler( )** - assigns the end element callback to the passed in parser object **XML_SetEmptyElementHandler( )** - assigns the empty element callback to the passed in parser object **XML_SetCharacterDataHandler( )** - assigns the character data callback to the passed in parser object **XML_SetProcessingInstructionHandler( )** - assigns the processing instruction callback to the passed in parser object **XML_SetCommentHandler( )** - assigns the comment handler callback to the passed in parser object **XML_SetCdataSectionHandler( )** - assigns the CDATA section start and end callbacks to the passed in parser object **XML_SetStartCdataSectionHandler( )** - assigns the start CDATA section callback to the passed in parser object **XML_SetEndCdataSectionHandler( )** - assigns the end CDATA section callback to the passed in parser object **XML_SetDefaultHandler( )** - assigns the default callback to the passed in parser object **XML_SetDefaultHandlerExpand( )** - assigns the default expand internal entities callback to the passed in parser object **XML_SetDoctypeDeclHandler( )** - assigns the start and end DOCTYPE declaration callbacks to the passed in parser object **XML_SetStartDoctypeDeclHandler( )** - assigns the start DOCTYPE declaration callback to the passed in parser object **XML_SetEndDoctypeDeclHandler( )** - assigns the end DOCTYPE declaration callback to the passed in parser object **XML_SetUnparsedEntityDeclHandler( )** - (OBSOLETE) assigns the unparsed entity declaration callback to the passed in parser object **XML_SetNotationDeclHandler( )** - assigns the NOTATION declaration callback to the passed in parser object **XML_SetNamespaceDeclHandler( )** - assigns the start and end namespace declaration callbacks to the passed in parser object **XML_SetStartNamespaceDeclHandler( )** - assigns the start namespace declaration callback to the passed in parser object **XML_SetEndNamespaceDeclHandler( )** - assigns the end namespace declaration callback to the passed in parser object **XML_SetNotStandaloneHandler( )** - assigns the not stand alone callback to the passed in parser object **XML_SetExternalEntityRefHandler( )** - assigns the external entity reference callback to the passed in parser object **XML_SetExternalEntityRefHandlerArg( )** - sets arg to be passed as the first argument to your external entity ref handler instead of the parser object **XML_SetSkippedEntityHandler( )** - assigns the skipped entity callback to the passed in parser object **XML_SetUnknownEncodingHandler( )** - assigns the unknown encoding callback to the passed in parser object **XML_SetElementDeclHandler( )** - assigns the ELEMENT declaration callback to the passed in parser object **XML_SetAttlistDeclHandler( )** - assigns the ATTLIST declaration callback to the passed in parser object **XML_SetEntityDeclHandler( )** - assigns the ENTITY declaration callback to the passed in parser object **XML_SetXmlDeclHandler( )** - assigns the XML declaration callback to the passed in parser object **XML_SetParamEntityParsing( )** - assigns the parameter entity parsing callback to the passed in parser object **XML_Parse( )** - parses input s, once isFinal is non-zero **XML_ParseBuffer( )** - parses the buffer stored within the parser object **XML_GetBuffer( )** - retrieves the parser object parse buffer **XML_GetErrorCode( )** - returns the latest parser error code **XML_GetCurrentByteIndex( )** - returns the current byte index that the parser is presently at within the document being parse

XML_GetCurrentByteCount( ) - return the number of bytes in the current event. Returns 0 if the event is in an internal entity **XML_GetInputContext( )** - returns the parser object buffer, as well as sets the size and offset argument variables **XML_GetCurrentLineNumber( )** - returns the current line that the parser is presently at within the document being parsed **XML_GetCurrentColumnNumber( )** - returns the current column that the parser is presently at within the document being parsed **XML_DefaultCurrent( )** - passes processing on to the default handler, but only for start element, end element, processing instruction and character data handlers **XML_ErrorString( )** - returns the corresponding error string for the passed in error code **XML_ExpatVersion( )** - returns a string containing the Expat verion number **XML_ExpatVersionInfo( )** - returns a structure containing the Expat version number **XML_IsWhiteSpace( )** - detects if the src buffer is whitespace

Description

This library provides XML parsing capability through a SAX based API.

This XML parser is based on the Expat XML parser (version 2.2.7), with the following value added features:

- XML 1.0 compliant - DTD parsing bugs have been fixed - source and external documentation of the parser where applicable - example code and tutorials - an empty element handler has been added - source has been made more human readable where applicable - VxWorks compatible - Wind River Workbench integration - interoperability testing on a variety of Windriver supported platforms

The XML parser is a stream-oriented parser in which an application registers callbacks for XML specific syntax events, for example, start tags, end tags, !ELEMENT declarations etc. The parser provides a large amount of flexibility allowing setup for such things as namespace handling, document encoding, parameter entity parsing, and external entity parsing. The parser also provides error handling, as well as the ability to pass user data throughout calls to your callbacks.

Include files

xmlConfig.h errno.h stddef.h string.h xmlLib.h xmltok.h xmlrole.h expat.h

Name

XML_ParserCreate( ) - creates a parser that is not concerned with namespaces or an external memory suite

Layer

EXPAT

Synopsis

```
XML_Parser XML_ParserCreate
    (
    const XML_Char *encodingName   /* '\0'-terminated string giving the name of the externall
y specified encoding */
    )
```

Description

This routine calls XML_ParserCreate_MM with a preset argument list that discludes a namespace separator, and a custom memory suite structure. XML_ParserCreate_MM constructs a new parser; encoding is the encoding specified by the external protocol, or **NULL** if there is none specified. encodingName, if it is being set, should be the encoding string specified in your document's XML declaration, i.e. <?xml version="1.0" encoding= "UTF-8"?>.

Returns

A pointer to the parser object.

Errno

N/A

See also

xmldoc

Name

XML_ParserCreateNS( ) - creates a parser that is not concerned with an external memory suite

Layer

EXPAT

Synopsis

```
XML_Parser XML_ParserCreateNS
    (
    const XML_Char *encodingName,   /* '\0'-terminated string giving the name of the externall
y specified encoding */
    XML_Char       nsSep            /* character to separate namespace from corresponding elem
ent */
    )
```

Description

This routine calls XML_ParserCreate_MM with a preset argument list that discludes a custom memory suite structure. XML_ParserCreate_MM constructs a new parser and namespace processor. Element type names and attribute names that belong to a namespace are expanded; unprefixed attribute names are never expanded; unprefixed element type names are expanded only if there is a default namespace. The expanded name is the concatenation of the namespace URI, the namespace separator character, and the local part of the name. If the namespace separator is **\0** then the namespace URI and the local part will be concatenated without any separator. When a namespace is not declared, the name and prefix will be passed through without expansion.

Returns

A pointer to the parser object.

Errno

N/A

See also

xmldoc

Name

XML_ParserCreate_MM( ) - general parser object setup and initialization

Layer

EXPAT

Synopsis

```
XML_Parser XML_ParserCreate_MM
    (
    const XML_Char            *encodingName,   /* '\0'-terminated string giving the name of th
e externally specified encoding */
    const XML_Memory_Handling_Suite *memsuite,       /* pointer to user defined memory suite s
tructure */
    const XML_Char            *nameSep         /* character to separate namespace from corresp
onding element */
    )
```

Description

This routine sets up and initializes the parser object. This involves allocating data buffers, setting up the memory handling routines, setting up how namespaces are displayed, and member variable initialization. In more detail, this routine constructs a new parser using the memory management suite referred to by memsuite. If memsuite is **NULL**, then the standard library memory suite is used. If namespaceSeparator is non-**NULL** it creates a parser with namespace processing as described above. The character pointed at will serve as the namespace separator.

All further memory operations used for the created parser will come from the given suite.

Returns

A pointer to the parser object.

Errno

N/A

See also

xmldoc

Name

XML_ParserReset( ) - re-initializes the whole parser object

Layer

EXPAT

Synopsis

```
XML_Bool XML_ParserReset
    (
    XML_Parser     parser,         /* parser object to reset */
    const XML_Char *encodingName   /* '\0'-terminated string giving the name of the externall
y specified encoding */
    )
```

Description

This routine frees all memory used by the present parser object, then re-initializes the object in preparation for re-use. This is particularly valuable when memory allocation overhead is disproportionatly high, such as when a large number of small documents need to be parsed. All handlers are cleared from the parser.

Returns

Returns 1 if everything succeeds, 0 if there is a failure.

Errno

N/A

See also

xmldoc

Name

XML_SetEncoding( ) - sets the encoding for the XML parser, indicating how the document to be parsed is encoded

Layer

EXPAT

Synopsis

```
int XML_SetEncoding
    (
    XML_Parser     parser,         /* parser object */
    const XML_Char *encodingName   /* '\0'-terminated string giving the name of the externall
y specified encoding */
    )
```

Description

This routine sets the encoding for the XML parser, indicating how the document to be parsed is encoded. It is equivalent to supplying an encoding argument to XML_ParserCreate. It must not be called after XML_Parse or XML_ParseBuffer.

Returns

1 for success, 0 for error.

Errno

N/A

See also

xmldoc

Name

XML_ExternalEntityParserCreate( ) - creates an XML_Parser object that can parse an external general entity

Layer

EXPAT

Synopsis

```
XML_Parser XML_ExternalEntityParserCreate
    (
    XML_Parser     oldParser,      /* parser object */
    const XML_Char *context,       /* '\0'-terminated string specifying the parse context */
    const XML_Char *encodingName   /* '\0'-terminated string giving the name of the externall
y specified encoding */
    )
```

Description

This routine creates an XML_Parser object that can parse an external general entity. It creates a new parser that retains all of the characteristics of the original document parser that is passed into this routine(i.e. handlers user date etc...). context is a **\0**-terminated string specifying the parse context; encoding is a **\0**-terminated string giving the name of the externally specified encoding, or **NULL** if there is no externally specified encoding. The context string consists of a sequence of tokens separated by formfeeds (\f); a token consisting of a name specifies that the general entity of the name is open; a token of the form prefix=uri specifies the namespace for a particular prefix; a token of the form =uri specifies the default namespace. This can be called at any point after the first call to an ExternalEntityRefHandler as long as the parser has not yet been freed. The new parser is completely independent and may safely be used in a separate thread. The handlers and userData are initialized from the parser argument.

Returns

Returns 0 if out of memory. Otherwise returns a new XML_Parser object.

Errno

N/A

See also

xmldoc

Name

XML_ParserFree( ) - frees all memory used by the parser object passed in

Layer

EXPAT

Synopsis

```
void XML_ParserFree
    (
    XML_Parser parser   /* parser object to free */
    )
```

Description

This routine frees all memory used by the parser object passed in.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_UseParserAsHandlerArg( ) - sets the m_handlerArg member variable of the parser passed in to be the parser

Layer

EXPAT

Synopsis

```
void XML_UseParserAsHandlerArg
    (
    XML_Parser parser   /* parser object to configure*/
    )
```

Description

This routine sets the m_handlerArg member variable of the parser passed in to be the parser. If this routine is called, then the parser will be passed as the first argument to callbacks instead of userData. The userData will still be accessible using XML_GetUserData.

Returns

N/A

Errno

S_xmlLib_CANT_CHANGE_FEATURE_ONCE_PARSING S_xmlLib_FEATURE_REQUIRES_XML_DTD

See also

xmldoc

Name

XML_UseForeignDTD( ) - forces the parser to assume that there is an external subset

Layer

EXPAT

Synopsis

```
enum XML_Error XML_UseForeignDTD
    (
    XML_Parser parser,   /* parser object to configure */
    XML_Bool   useDTD    /* force foreign DTD to be used */
    )
```

Description

This routine forces the parser to assume that there is an external subset If useDTD == **XML_TRUE** is passed to this routine, then the parser will assume that there is an external subset, even if none is specified in the document. In such a case the parser will call the externalEntityRefHandler with a value of **NULL** for the systemId argument (the publicId and context arguments will be **NULL** as well).

Note: If this routine is called, it must be done before the first call to XML_Parse or XML_ParseBuffer. It will have no effect after that. Returns **XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING**.

Note: If the document does not have a DOCTYPE declaration at all, then startDoctypeDeclHandler and endDoctypeDeclHandler will not be called, despite an external subset being parsed.

Note: If **XML_DTD** is not defined when Expat is compiled, returns **XML_ERROR_FEATURE_REQUIRES_XML_DTD**.

Returns

Returns an XML_Error enumeration, **XML_ERROR_NONE** if **OK**, otherwise an enumerated error.

Errno

S_xmlLib_CANT_CHANGE_FEATURE_ONCE_PARSING S_xmlLib_FEATURE_REQUIRES_XML_DTD

See also

xmldoc

Name

XML_SetReturnNSTriplet( ) - sets how namespace triplets are displayed, e.g. URI + seperator + local_name

Layer

EXPAT

Synopsis

```
void XML_SetReturnNSTriplet
    (
    XML_Parser parser,   /* parser object to configure */
    int        do_nst    /* cause display of names as URI + sep + local_name */
    )
```

Description

This routine sets how namespace triplets are displayed, e.g. URI + seperator + local_name. If do_nst is non-zero, and namespace processing is in effect, and a name has a prefix (i.e. an explicit namespace qualifier) then that name is returned as a triplet in a single string separated by the separator character specified when the parser was created: URI + sep + local_name + sep + prefix. If do_nst is zero, then namespace information is returned in the default manner (URI + sep + local_name) whether or not the name has a prefix.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetUserData( ) - assigns an arbitrary piece of data/structure to be passed throughout routines as userData

Layer

EXPAT

Synopsis

```
void XML_SetUserData
    (
    XML_Parser parser,   /* parser object to configure */
    void       *p        /* arbitrary data to set as userData */
    )
```

Description

This routine assigns an arbitrary piece of data/structure to be passed throughout routines as userData. This is done by setting the m_userData member variable of the parser passed in to be the passed in p variable. This allows whatever p is to be passed throughout handler routines as the userData variable.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetBase( ) - sets the base directory of the parser object, for relative paths used in entity handling

Layer

EXPAT

Synopsis

```
int XML_SetBase
    (
    XML_Parser     parser,   /* parser object to configure */
    const XML_Char *p        /* base path string */
    )
```

Description

This routine copies the base path p into the parser member variable m_curBase. In general, it sets the base to be used for resolving relative URIs in system identifiers in declarations. Resolving relative identifiers is left to the application: this value will be passed through as the base argument to the XML_ExternalEntityRefHandler, XML_NotationDeclHandler, and XML_UnparsedEntityDeclHandler. The base argument will be copied.

Returns

Returns zero if out of memory, non-zero otherwise.

Errno

N/A

See also

xmldoc

Name

XML_GetBase( ) - retrieves the base path stored in the parser object

Layer

EXPAT

Synopsis

```
const XML_Char *XML_GetBase
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine retrieves the m_curBase member variable of the parser. This variable is used for external entity relative paths during entity parsing.

Returns

A pointer to the m_curBase member variable of the parser object.

Errno

N/A

See also

xmldoc

Name

XML_GetSpecifiedAttributeCount( ) - returns the number of the attribute/value pairs specified in the last element, not including default attributes

Layer

EXPAT

Synopsis

```
int XML_GetSpecifiedAttributeCount
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns the number of the attribute/value pairs passed in last call to the XML_StartElementHandler that were specified in the start-tag rather than defaulted. Each attribute/value pair counts as 2; this corresponds to an index into the atts array passed to the XML_StartElementHandler.

Returns

parser->nSpecifiedAtts variable value.

Errno

N/A

See also

xmldoc

Name

XML_GetIdAttributeIndex( ) - retrieves the ID attribute index passed in the last call to XML_StartElementHandler

Layer

EXPAT

Synopsis

```
int XML_GetIdAttributeIndex
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns the index of the ID attribute passed in the last call to XML_StartElementHandler, or -1 if there is no ID attribute. Each attribute/value pair counts as 2; this corresponds to an index into the atts array passed to the XML_StartElementHandler.

Returns

parser->m_idAttIndex variable value.

Errno

N/A

See also

xmldoc

Name

XML_SetElementHandler( ) - assigns the start and end element callbacks to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetElementHandler
    (
    XML_Parser              parser,   /* parser object to configure */
    XML_StartElementHandler start,    /* function pointer to start element handler */
    XML_EndElementHandler   end       /* function pointer to end element handler */
    )
```

Description

This routine sets the start and end element callback function pointers in the passed in parser object. These callbacks are later called for every start and end element event.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetStartElementHandler( ) - assigns the start element callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetStartElementHandler
    (
    XML_Parser              parser,   /* parser object to configure */
    XML_StartElementHandler start     /* function pointer to start element handler */
    )
```

Description

This routine sets the start element callback function pointer in the passed in parser object. This callback is later called for every start element event.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetEndElementHandler( ) - assigns the end element callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetEndElementHandler
    (
    XML_Parser            parser,   /* parser object to configure */
    XML_EndElementHandler end       /* function pointer to end element handler */
    )
```

Description

This routine sets the end element callback function pointer in the passed in parser object. This callback is later called for every end element event.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetEmptyElementHandler( ) - assigns the empty element callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetEmptyElementHandler
    (
    XML_Parser              parser,   /* parser object to configure */
    XML_EmptyElementHandler empty     /* function pointer to empty element handler */
    )
```

Description

This routine sets the empty element callback function pointer in the passed in parser object. This callback is called whenever a empty element is encountered, i.e. <element /> which is also equivalent to <element></element>.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetCharacterDataHandler( ) - assigns the character data callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetCharacterDataHandler
    (
    XML_Parser               parser,   /* parser object to configure */
    XML_CharacterDataHandler handler   /* function pointer to character data handler */
    )
```

Description

This routine sets the character data callback function pointer in the passed in parser object. This callback is called whenever element character content is encountered.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetProcessingInstructionHandler( ) - assigns the processing instruction callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetProcessingInstructionHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_ProcessingInstructionHandler handler   /* function pointer to processing instruction h
andler */
    )
```

Description

This routine sets the processing instruction callback function pointer in the passed in parser object. This callback is called whenever a processing instruction is encountered.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetCommentHandler( ) - assigns the comment handler callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetCommentHandler
    (
    XML_Parser         parser,   /* parser object to configure */
    XML_CommentHandler handler   /* function pointer to comment handler */
    )
```

Description

This routine sets the comment handler callback function pointer in the passed in parser object. This callback is called whenever a comment is encountered in the XML document.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetCdataSectionHandler( ) - assigns the CDATA section start and end callbacks to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetCdataSectionHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_StartCdataSectionHandler start,    /* function pointer to start CDATA section handler
*/
    XML_EndCdataSectionHandler end       /* function pointer to end CDATA section handler */
    )
```

Description

This routine sets the CDATA section start and end callback function pointers in the passed in parser object. This callback is called whenever a CDATA section is encountered. It is important to note that CDATA is different from the character data. Character data refers to text within elements.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetStartCdataSectionHandler( ) - assigns the start CDATA section callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetStartCdataSectionHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_StartCdataSectionHandler start     /* function pointer to start CDATA section handler
*/
    )
```

Description

This routine sets the start CDATA section callback function pointer in the passed in parser object. This callback is called whenever the start of a CDATA section is encountered.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetEndCdataSectionHandler( ) - assigns the end CDATA section callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetEndCdataSectionHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_EndCdataSectionHandler end       /* function pointer to end CDATA section handler */
    )
```

Description

This routine sets the end CDATA section callback function pointer in the passed in parser object. This callback is called whenever the end of a CDATA section is encountered.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetDefaultHandler( ) - assigns the default callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetDefaultHandler
    (
    XML_Parser         parser,   /* parser object to configure */
    XML_DefaultHandler handler   /* function pointer to default handler */
    )
```

Description

This routine sets the default handler and also inhibits expansion of internal entities. These entity references will be passed to the default handler, or to the skipped entity handler, if one is set.

The callback is called for any characters in the XML document for which there is no applicable handler. This includes characters that are part of markup which is of a kind that is not reported (comments, markup declarations), and characters that are part of a construct which could be reported but for which no handler has been supplied. The characters are passed exactly as they were in the XML document except that they are encoded in UTF-8. Line boundaries are not normalized. Note that a byte order mark character is not passed to the default handler. There are no guarantees about how characters are divided between calls to the default handler. For example, a comment might be split between multiple calls.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetDefaultHandlerExpand( ) - assigns the default expand internal entities callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetDefaultHandlerExpand
    (
    XML_Parser         parser,   /* parser object to configure */
    XML_DefaultHandler handler   /* function pointer to default handler */
    )
```

Description

This routine sets the default handler but does not inhibit expansion of internal entities. The entity reference is not passed to the default handler. This callback is called whenever an XML event occurs and there is no specific callback already assigned for that event. For example, if no callbacks are set, the default handler is the only callback called, if it is set.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetDoctypeDeclHandler( ) - assigns the start and end DOCTYPE declaration callbacks to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetDoctypeDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_StartDoctypeDeclHandler start,    /* function pointer to start DOCTYPE declaration han
dler */
    XML_EndDoctypeDeclHandler end       /* function pointer to end DOCTYPE declaration handle
r */
    )
```

Description

This routine sets the start and end DOCTYPE declaraction callback function pointers in the passed in parser object. The start is called for the start of the DOCTYPE declaration, before any DTD or internal subset is parsed. The end is called for the start of the DOCTYPE declaration when the closing > is encountered, but after processing any external subset.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetStartDoctypeDeclHandler( ) - assigns the start DOCTYPE declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetStartDoctypeDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_StartDoctypeDeclHandler start     /* function pointer to start DOCTYPE declaration han
dler */
    )
```

Description

This routine sets the start DOCTYPE declaration callback function pointer in the passed in parser object. The start is called for the start of the DOCTYPE declaration, before any DTD or internal subset is parsed.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetEndDoctypeDeclHandler( ) - assigns the end DOCTYPE declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetEndDoctypeDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_EndDoctypeDeclHandler end       /* function pointer to end DOCTYPE declaration handle
r */
    )
```

Description

This routine sets the end DOCTYPE declaration callback function pointer in the passed in parser object. The end is called for the start of the DOCTYPE declaration when the closing > is encountered, but after processing any external subset.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetUnparsedEntityDeclHandler( ) - (OBSOLETE) assigns the unparsed entity declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetUnparsedEntityDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_UnparsedEntityDeclHandler handler   /* function pointer to unparsed entity declaratio
n handler */
    )
```

Obsolete -- obsolete -- obsolete

This routine sets the unparsed entity decaration callback function pointer in the passed in parser object. This handler has been superceded by the EntityDeclHandler. It is provided here for backward compatibility. The callback is called for a declaration of an unparsed (NDATA) entity.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetNotationDeclHandler( ) - assigns the NOTATION declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetNotationDeclHandler
    (
    XML_Parser              parser,   /* parser object to configure */
    XML_NotationDeclHandler handler   /* function pointer to NOTATION declaration handler */
    )
```

Description

This routine sets the NOTATION declaration callback function pointer in the passed in parser object. The callback is called for a declaration of NOTATION.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetNamespaceDeclHandler( ) - assigns the start and end namespace declaration callbacks to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetNamespaceDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_StartNamespaceDeclHandler start,    /* function pointer to start namespace declaratio
n handler */
    XML_EndNamespaceDeclHandler end       /* function pointer to end namespace declaration han
dler */
    )
```

Description

This routine sets the start and end namespace declaration callback function pointers in the passed in parser object. When namespace processing is enabled, these callbacks are called once for each namespace declaration. The call to the start and end element handlers occur between the calls to the start and end namespace declaration handlers.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetStartNamespaceDeclHandler( ) - assigns the start namespace declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetStartNamespaceDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_StartNamespaceDeclHandler start     /* function pointer to start namespace declaratio
n handler */
    )
```

Description

This routine sets the start namespace declaration callback function pointer in the passed in parser object. When namespace processing is enabled, this callback is called once for each namespace declaration. The call to the start and end element handlers occur between the calls to the start and end namespace declaration handlers.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetEndNamespaceDeclHandler( ) - assigns the end namespace declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetEndNamespaceDeclHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_EndNamespaceDeclHandler end       /* function pointer to end namespace declaration han
dler */
    )
```

Description

This routine sets the end namespace declaration callback function pointer in the passed in parser object. When namespace processing is enabled, this callback is called once for each namespace declaration. The call to the start and end element handlers occur between the calls to the start and end namespace declaration handlers.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetNotStandaloneHandler( ) - assigns the not stand alone callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetNotStandaloneHandler
    (
    XML_Parser               parser,   /* parser object to configure */
    XML_NotStandaloneHandler handler   /* function pointer to notstandalone handler */
    )
```

Description

This routine sets the not standalone callback function pointer in the passed in parser object. This callback is called if the document is not standalone (it has an external subset or a reference to a parameter entity, but does not have standalone="yes").

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetExternalEntityRefHandler( ) - assigns the external entity reference callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetExternalEntityRefHandler
    (
    XML_Parser                parser,   /* parser object to configure */
    XML_ExternalEntityRefHandler handler   /* function pointer to external entity reference ha
ndler */
    )
```

Description

This routine sets the external entity reference callback function pointer in the passed in parser object. This callback is called whenever an external entity is referenced in the parsed XML document.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetExternalEntityRefHandlerArg( ) - sets arg to be passed as the first argument to your external entity ref handler instead of the parser object

Layer

EXPAT

Synopsis

```
void XML_SetExternalEntityRefHandlerArg
    (
    XML_Parser parser,   /* parser object to configure */
    void       *arg      /* arbitrary data to be set as external entity reference user data */
    )
```

Description

This routine sets arg to be passed as the first argument to your external entity ref handler instead of the parser object. If a non-**NULL** value for arg is specified here, then it will be passed as the first argument to the external entity ref handler instead of the parser object.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetSkippedEntityHandler( ) - assigns the skipped entity callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetSkippedEntityHandler
    (
    XML_Parser               parser,   /* parser object to configure */
    XML_SkippedEntityHandler handler   /* function pointer to skipped entity handler */
    )
```

Description

This routine sets the skipped entity callback function pointer in the passed in parser object. This callback is called in two situations: 1) An entity reference is encountered for which no declaration has been read *and* this is not an error.
2) An internal entity reference is read, but not expanded, because XML_SetDefaultHandler has been called.
Note: Skipped parameter entities in declarations and skipped general entities in attribute values cannot be reported because the event would be out of sync with the reporting of the declarations or attribute values .

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetUnknownEncodingHandler( ) - assigns the unknown encoding callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetUnknownEncodingHandler
    (
    XML_Parser                parser,    /* parser object to configure */
    XML_UnknownEncodingHandler handler,   /* function pointer to unknown encoding handler */
    void                      *data
    )
```

Description

This routine sets the unknown encoding callback function pointer in the passed in parser object. This callback is called for an encoding that is unknown to the parser.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetElementDeclHandler( ) - assigns the ELEMENT declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetElementDeclHandler
    (
    XML_Parser             parser,   /* parser object to configure */
    XML_ElementDeclHandler eldecl    /* function pointer to ELEMENT declaration handler */
    )
```

Description

This routine sets the ELEMENT declaration callback function pointer in the passed in parser object. This callback is called for an element declaration.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetAttlistDeclHandler( ) - assigns the ATTLIST declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetAttlistDeclHandler
    (
    XML_Parser             parser,   /* parser object to configure */
    XML_AttlistDeclHandler attdecl   /* function pointer to ATTLIST declaration handler */
    )
```

Description

This routine sets the ATTLIST declaration callback function pointer in the passed in parser object. The ATTLIST declaration handler is called for *each* attribute. A single Attlist declaration with multiple attributes declared will generate multiple calls to this handler.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetEntityDeclHandler( ) - assigns the ENTITY declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetEntityDeclHandler
    (
    XML_Parser            parser,   /* parser object to configure */
    XML_EntityDeclHandler handler   /* function pointer to ENTITY declaration handler */
    )
```

Description

This routine sets the ENTITY declaration callback function pointer in the passed in parser object. This is called for entity declarations.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetXmlDeclHandler( ) - assigns the XML declaration callback to the passed in parser object

Layer

EXPAT

Synopsis

```
void XML_SetXmlDeclHandler
    (
    XML_Parser         parser,   /* parser object to configure */
    XML_XmlDeclHandler handler   /* function pointer to xml declaration handler */
    )
```

Description

This routine sets the XML declaration callback function pointer in the passed in parser object. The XML declaration handler is called for *both* XML declarations and text declarations.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_SetParamEntityParsing( ) - assigns the parameter entity parsing callback to the passed in parser object

Layer

EXPAT

Synopsis

```
int XML_SetParamEntityParsing
    (
    XML_Parser                parser,     /* parser object to configure */
    enum XML_ParamEntityParsing peParsing   /* desired parameter entity parsing mode */
    )
```

Description

This routine sets the parameter entity parsing callback function pointer in the passed in parser object. It controls parsing of parameter entities (including the external DTD subset). If parsing of parameter entities is enabled, then references to external parameter entities (including the external DTD subset) are passed to the handler set with XML_SetExternalEntityRefHandler. The context passed is 0.

Unlike external general entities, external parameter entities can only be parsed synchronously. If the external parameter entity is to be parsed, it must be parsed during the call to the external entity ref handler. The complete sequence of XML_ExternalEntityParserCreate, XML_Parse/XML_ParseBuffer and XML_ParserFree calls must be made during this call. After XML_ExternalEntityParserCreate has been called to create the parser for the external parameter entity (context must be 0 for this call), it is illegal to make any calls on the old parser until XML_ParserFree has been called on the newly created parser. If the library has been compiled without support for parameter entity parsing (without **XML_DTD** being defined), then XML_SetParamEntityParsing will return 0 if parsing of parameter entities is requested; otherwise it will return non-zero.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_Parse( ) - parses input s, once isFinal is non-zero

Layer

EXPAT

Synopsis

```
enum XML_Status XML_Parse
    (
    XML_Parser parser,   /* parser object to parse with */
    const char *s,       /* buffer containing xml document to parse */
    int        len,      /* length of s */
    int        isFinal   /* indicate if this is the final piece of the xml document being pass
ed in */
    )
```

Description

This routine takes in the document either in pieces or all at once. Once the document has been completely passed the last call to XML_Parse must have isFinal true; len may be zero for this call (or any other). The document is parsed as it is passed in, meaning, if it is passed in pieces then each piece is parsed as it is passed, and if it is passed in whole then the document is parsed as a whole. The way it is passed in has no influence on the final parse result. When XML_Parse is called the document or document piece being passed into that routine is copied into a buffer within the parser object. The buffer that contains the document or document piece is then retrieved using XML_GetBuffer to retrieve a pointer to the buffer, and parsed by calling XML_ParseBuffer.

Returns

Returns 0 if a fatal error is detected, non-zero otherwise

Errno

S_xmlLib_NO_MEMORY

See also

xmldoc

Name

XML_ParseBuffer( ) - parses the buffer stored within the parser object

Layer

EXPAT

Synopsis

```
enum XML_Status XML_ParseBuffer
    (
    XML_Parser parser,   /* parser object to parse */
    int        len,      /* length of the buffer to parse */
    int        isFinal   /* indicate if this is the final piece of buffer to parse */
    )
```

Description

This routine performs the XML parsing on the buffer in the parser object containing the document or document piece being parsed. This is called by XML_Parse, and should not be called outside of that routine.

Returns

Returns 0 if a fatal error is detected, non-zero otherwise

Errno

N/A

See also

xmldoc

Name

XML_GetBuffer( ) - retrieves the parser object parse buffer

Layer

EXPAT

Synopsis

```
void *XML_GetBuffer
    (
    XML_Parser parser,   /* parser object to retrieve from */
    int        len       /* length of buffer to retrieve */
    )
```

Description

This routine retrieves a void pointer to the parser object parse buffer.

Returns

N/A

Errno

S_xmlLib_NO_MEMORY

See also

xmldoc

Name

XML_GetErrorCode( ) - returns the latest parser error code

Layer

EXPAT

Synopsis

```
enum XML_Error XML_GetErrorCode
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns the latest parser error code.

Returns

Returns XML_Error enumeration, that has a corresponding error string.

Errno

N/A

See also

xmldoc

Name

XML_GetCurrentByteIndex( ) - returns the current byte index that the parser is presently at within the document being parse

Layer

EXPAT

Synopsis

```
XML_Index XML_GetCurrentByteIndex
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns information about the current parse location. It may be called when XML_Parse or XML_ParseBuffer return 0; in this case the location is the location of the character at which the error was detected. This may also be called from any other callback called to report some parse event; in this the location is the location of the first of the sequence of characters that generated the event.

Returns

Returns the current byte index that the parser is presently at within the document being parsed.

Errno

N/A

See also

xmldoc

Name

XML_GetCurrentByteCount( ) - return the number of bytes in the current event. Returns 0 if the event is in an internal entity

Layer

EXPAT

Synopsis

```
int XML_GetCurrentByteCount
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns the number of bytes in the current event. Returns 0 if the event is in an internal entity.

Returns

Return the number of bytes in the current event. Returns 0 if the event is in an internal entity.

Errno

N/A

See also

xmldoc

Name

XML_GetInputContext( ) - returns the parser object buffer, as well as sets the size and offset argument variables

Layer

EXPAT

Synopsis

```
const char *XML_GetInputContext
    (
    XML_Parser parser,    /* parser object to retrieve from */
    int        *offset,
    int        *size
    )
```

Description

This routine returns the parser object buffer, as well as sets the size and offset argument variables. If **XML_CONTEXT_BYTES** is defined, returns the input buffer, sets the integer pointed to by offset to the offset within this buffer of the current parse position, and sets the integer pointed to by size to the size of this buffer (the number of input bytes). Otherwise returns a **NULL** pointer. Also returns a **NULL** pointer if a parse isn't active.

Note

The character pointer returned should not be used outside the handler that makes the call.

Returns

Returns the parser object buffer (m_buffer member variable).

Errno

N/A

See also

xmldoc

Name

XML_GetCurrentLineNumber( ) - returns the current line that the parser is presently at within the document being parsed

Layer

EXPAT

Synopsis

```
XML_Size XML_GetCurrentLineNumber
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns information about the current parse location. It may be called when XML_Parse or XML_ParseBuffer return 0; in this case the location is the location of the character at which the error was detected. This may also be called from any other callback called to report some parse event; in this the location is the location of the first of the sequence of characters that generated the event.

Returns

Returns the current line that the parser is at within the document being parsed.

Errno

S_xmlLib_UNKNOWN_ENCODING

See also

xmldoc

Name

XML_GetCurrentColumnNumber( ) - returns the current column that the parser is presently at within the document being parsed

Layer

EXPAT

Synopsis

```
XML_Size XML_GetCurrentColumnNumber
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine returns information about the current parse location. It may be called when XML_Parse or XML_ParseBuffer return 0; in this case the location is the location of the character at which the error was detected. This may also be called from any other callback called to report some parse event; in this the location is the location of the first of the sequence of characters that generated the event.

Returns

Returns the current column that the parser is at within the document being parsed

Errno

S_xmlLib_UNKNOWN_ENCODING

See also

xmldoc

Name

XML_DefaultCurrent( ) - passes processing on to the default handler, but only for start element, end element, processing instruction and character data handlers

Layer

EXPAT

Synopsis

```
void XML_DefaultCurrent
    (
    XML_Parser parser   /* parser object to retrieve from */
    )
```

Description

This routine passes processing on to the default handler, but only for start element, end element, processing instruction and character data handlers. This can be called within a handler for a start element, end element, processing instruction or character data. It causes the corresponding markup to be passed to the default handler.

Returns

N/A

Errno

N/A

See also

xmldoc

Name

XML_ErrorString( ) - returns the corresponding error string for the passed in error code

Layer

EXPAT

Synopsis

```
const XML_LChar *XML_ErrorString
    (
    enum XML_Error code   /* Expat SAX XML parser specific error code to decode */
    )
```

Description

This routine returns the corresponding error string for the passed in error code.

Returns

Returns the corresponding error string for the passed in error code.

Errno

N/A

See also

xmldoc

Name

XML_ExpatVersion( ) - returns a string containing the Expat verion number

Layer

EXPAT

Synopsis

```
const XML_LChar * XML_ExpatVersion(void)
```

Description

This routine returns a string containing the Expat version number.

Returns

Returns a string containing the Expat version number.

Errno

N/A

See also

xmldoc

Name

XML_ExpatVersionInfo( ) - returns a structure containing the Expat version number

Layer

EXPAT

Synopsis

```
XML_Expat_Version XML_ExpatVersionInfo(void)
```

Description

This routine returns a structure containing the Expat version number.

Returns

Returns a structure containing the Expat version number.

Errno

N/A

See also

xmldoc

Name

XML_IsWhiteSpace( ) - detects if the src buffer is whitespace

Layer

EXPAT

Synopsis

```
XML_Bool XML_IsWhiteSpace
    (
    const char *src,
    int        len
    )
```

Description

This routine detects if the src buffer is whitespace.

Returns

XML_TRUE if src is whitespace, **XML_FALSE** otherwise.

Errno

N/A

See also

xmldoc

3. XMLOP

Name

xmlop - simple API for XML output

Layer

EXPAT

Routines

xmlOutputInit( ) - dummy init routine **XML_OutputCreate( )** - create an XML output object **XML_OutputReset( )** - resets an output object **XML_OutputDestroy( )** - destroys an XML output object **XML_OutputFormatSet( )** - turns output formatting on/off **XML_OutputCanonSet( )** - turns output canonicalization formatting on/off **XML_OutputSecondCanonFormSet( )** - turns output 2nd canonicalization form formatting on/off **XML_XmlDeclWrite( )** - write an XML (or text) declaration **XML_StartNamespaceDeclWrite( )** - begins a namespace **XML_EndNamespaceDeclWrite( )** - ends a namespace **XML_StartElementWrite( )** - writes an element start tag **XML_EndElementWrite( )** - writes an element end tag **XML_CharacterDataWrite( )** - write arbitrary character data **XML_ProcessingInstructionWrite( )** - writes an XML processing instruction **XML_CommentWrite( )** - writes an XML comment **XML_StartCdataSectionWrite( )** - begins a CDATA section **XML_EndCdataSectionWrite( )** - ends a CDATA section **XML_EntityRefWrite( )** - writes an entity reference **XML_StartDoctypeDeclWrite( )** - starts a doctype declaration, does not print immediately if the Canonicalization option is turned on **XML_EndDoctypeDeclWrite( )** - ends a doctype declaration **XML_EntityDeclWrite( )** - writes a complete entity declaration **XML_NotationDeclWrite( )** - write a complete notation declaration, this does not print immediately if the Canonicalization option is turned on **XML_AttlistDeclWrite( )** - writes a complete attlist declaration **XML_ElementDeclWrite( )** - writes a complete element declaration

Description

This library provides a SAX-like API for outputting XML to a file pointer. For example the routine for handling an XML start element in SAX is XML_StartElementHandler, and in this output module to write an XML start element you call the routine XML_StartElementWrite.

Includes

stdio.h, **stdlib.h**, **errnoLib.h**, **vxworks.h**, **xmlop.h**, **expat.h**, **xmltchar.h**

Name

xmlOutputInit( ) - dummy init routine

Layer

EXPAT

Synopsis

```
void xmlOutputInit(void)
```

Description

This routine is a dummy init routine for Tornado component inclusion/dependency checking. Currently does nothing.

Returns

N/A

Errno

N/A

See also

xmlop

Name

XML_OutputCreate( ) - create an XML output object

Layer

EXPAT

Synopsis

```
XML_Output XML_OutputCreate
    (
    const XML_Char * encoding,   /* output char encoding, not presently used, exists only to m
atch the SAX API */
    XML_Char         nsSep,      /* namespace separator char */
    FILE *           fp          /* output file pointer */
    )
```

Description

This routine constructs a new output object. encoding is the output stream encoding; if encoding is "UTF-16", it is the responsibility of the user to write the byte order mark prior to any other output. nsSep is the namespace/name separator character; if the output object is being used in conjunction with the expat parser, this must be set to the same value as the parser's namespace separator. Avoid using ":" as the separator, as some output routines may misinterpret names given in the format "prefix:tagname". Output is written to fp.

Returns

A new XML_Output object, or **NULL** if one cannot be created.

Errno

S_memLib_NOT_ENOUGH_MEMORY

See also

xmlop, **XML_StartElementWrite( )**

Name

XML_OutputReset( ) - resets an output object

Layer

EXPAT

Synopsis

```
STATUS XML_OutputReset
    (
    XML_Output       output,     /* XML output object */
    const XML_Char * encoding,   /* output char encoding */
    XML_Char         nsSep,      /* namespace separator char */
    FILE *           fp          /* new output stream */
    )
```

Description

This routine prepares an output object to be re-used. This is particularly valuable when memory allocation overhead is disproportionatley high, such as when a large number of small documents need to be created. All namespace declarations and tag stacks are cleared. encoding, nsSep and fp are as described in **XML_OutputCreate( )**.

Returns

OK

Errno

N/A

See also

xmlop, **XML_OutputCreate( )**

Name

XML_OutputDestroy( ) - destroys an XML output object

Layer

EXPAT

Synopsis

```
STATUS XML_OutputDestroy
    (
    XML_Output output   /* XML output object */
    )
```

Description

This routine destroys an existing output object. All memory used by output is freed. The stream passed to **XML_OutputCreate( )** is NOT closed; closing of the stream is left to the user.

Returns

OK

Errno

N/A

See also

xmlop, **XML_OutputCreate( )**

Name

XML_OutputFormatSet( ) - turns output formatting on/off

Layer

EXPAT

Synopsis

```
STATUS XML_OutputFormatSet
    (
    XML_Output output,   /* XML output object */
    XML_Bool   format    /* enable/disable formatting */
    )
```

Description

This routine turns on/off automatic formatting for output. Tags are indented based on depth, and a newline is placed before each start tag and after each end tag. This is not related to Canonical formatting.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_OutputCanonSet( ) - turns output canonicalization formatting on/off

Layer

EXPAT

Synopsis

```
STATUS XML_OutputCanonSet
    (
    XML_Output output,        /* XML output object */
    XML_Bool   canonicalize   /* enable/disable formatting */
    )
```

Description

This routine turns on/off automatic canonicalization formatting for output. Canonicalization of output will take place according to Canonical XML Version 1.0 specs at http://www.w3.org/TR/xml-c14n.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_OutputSecondCanonFormSet( ) - turns output 2nd canonicalization form formatting on/off

Layer

EXPAT

Synopsis

```
STATUS XML_OutputSecondCanonFormSet
    (
    XML_Output output,      /* XML output object */
    XML_Bool   secondForm   /* enable/disable formatting */
    )
```

Description

This routine turns on/off automatic 2nd form canonicalization formatting for output. Canonicalization of output will take place according to Canonical XML Version 1.0 specs at http://www.w3.org/TR/xml-c14n, with the exeption that DOCTYPE and NOTATION declarations are printed. The term 2nd Canonical form was coined by James Clark, for the purpose of the XML conformance test suite.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_XmlDeclWrite( ) - write an XML (or text) declaration

Layer

EXPAT

Synopsis

```
STATUS XML_XmlDeclWrite
    (
    XML_Output       output,      /* XML output object */
    const XML_Char * version,     /* XML specification version */
    const XML_Char * encoding,    /* document char. encoding */
    int              standalone   /* standalone flag */
    )
```

Description

This routine is called for both XML declarations and text declarations. The way to distinguish between the two is that the version parameter may be **NULL** for text declarations. The encoding parameter may be **NULL** for XML declarations. The standalone parameter may be -1, 0, or 1 indicating respectively that there is no standalone parameter in the declaration, that it is given as no, or that it is given as yes.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_StartNamespaceDeclWrite( ) - begins a namespace

Layer

EXPAT

Synopsis

```
STATUS XML_StartNamespaceDeclWrite
    (
    XML_Output       output,   /* XML output object */
    const XML_Char * prefix,   /* namespace prefix (local name) */
    const XML_Char * uri       /* namespace URI */
    )
```

Description

This routine is called once for each namespace declaration; the namespace declaration will be generated as output in the subsequent start tag. Calls to **XML_StartElementWrite( )** and **XML_EndElementWrite( )** occur between the calls to the start and end namespace declaration routines. For an xmlns attribute, prefix is **NULL**. For an xmlns="" attribute, uri is **NULL**.

Returns

ERROR in the event of a namespace stack or name buffer overflow, **OK** otherwise.

Errno

S_xmlop_NSSTACK_OVERFLOW S_xmlop_NAMEBUF_CONCAT

See also

xmlop, **XML_StartElementWrite( )**, **XML_EndElementWrite( )**

Name

XML_EndNamespaceDeclWrite( ) - ends a namespace

Layer

EXPAT

Synopsis

```
STATUS XML_EndNamespaceDeclWrite
    (
    XML_Output       output,   /* XML output object */
    const XML_Char * prefix    /* namespace prefix (local name) */
    )
```

Description

This routine is called once to end each namespace declaration. The declaration to end a namespace does not affect the output and is not necessary. This routine is only present to match the SAX API.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_StartElementWrite( ) - writes an element start tag

Layer

EXPAT

Synopsis

```
STATUS XML_StartElementWrite
    (
    XML_Output        output,   /* XML output object */
    const XML_Char *  name,     /* element tagname */
    const XML_Char ** atts      /* element attributes */
    )
```

Description

This routine writes an element start tag with the given name, the list of namespace declarations established in prior calls to **XML_StartNamespaceDeclWrite( )**, and the list of attributes given by atts.

name is of the format "http://www.some.uri/nsuri!-- --§tagname", where the URI is the fully qualified URI for the desired namespace, as passed to **XML_StartNamespaceDeclWrite( )**, "§" is the namespace separator char passed to **XML_OutputCreate( )**b and "tagname" is the base name of the element.

atts is an array of attribute name/value pairs, terminated with a **NULL** sentinel. Attribute names and values are **NULL** terminated strings; attribute names are in the format described above.

If no namespaces are to be declared in this tag, names may be specified without the URI or §. As a shortcut if the output object is not being used to process input from a parser, namespace delimited tag and attribute names may be passed in in the form "prefix:name".

Returns

OK

Errno

N/A

See also

xmlop, **XML_StartNamespaceDeclWrite( )**

Name

XML_EndElementWrite( ) - writes an element end tag

Layer

EXPAT

Synopsis

```
STATUS XML_EndElementWrite
    (
    XML_Output       output,   /* XML output object */
    const XML_Char * name      /* element tagname */
    )
```

Description

This routine writes an element end tag. name is in the format described in **XML_StartElementWrite( )**.

Note that this routine does not check that name agrees with that of the corresponding element start tag.

Returns

OK

Errno

N/A

See also

xmlop, **XML_StartElementWrite( )**

Name

XML_CharacterDataWrite( ) - write arbitrary character data

Layer

EXPAT

Synopsis

```
STATUS XML_CharacterDataWrite
    (
    XML_Output       output,   /* XML output object */
    const XML_Char * s,        /* char data (non NULL-terminated) */
    int              len       /* length of char data */
    )
```

Description

This routine writes len bytes of the string s to the output. Note that s is not **NULL**-terminated. The characters **&**, **<**, **>**, are expanded into entity refernces unless a CDATA section is active. If a CDATA section is active, the entire string is written literally.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_ProcessingInstructionWrite( ) - writes an XML processing instruction

Layer

EXPAT

Synopsis

```
STATUS XML_ProcessingInstructionWrite
    (
    XML_Output       output,   /* XML output object */
    const XML_Char * target,   /* PI target */
    const XML_Char * data      /* PI data */
    )
```

Description

This routine writes an XML processing instruction with the specified target and data. target and data are both **NULL**-terminated, and either or both may be **NULL**. If data is **NULL**, a PI with only a target is written; if target is **NULL**, an empty PI is written, regardless of the value of data.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_CommentWrite( ) - writes an XML comment

Layer

EXPAT

Synopsis

```
STATUS XML_CommentWrite
    (
    XML_Output       output,    /* XML output object */
    const XML_Char * data       /* comment text (NULL-terminated) */
    )
```

Description

This routine writes an XML comment to the output, where data is **NULL**-terminated and provides the complete comment text.

Returns

ERROR if data contains "-->" or ends with "-", **OK** otherwise.

Errno

S_xmlop_ILLEGAL_STRING

See also

xmlop

Name

XML_StartCdataSectionWrite( ) - begins a CDATA section

Layer

EXPAT

Synopsis

```
STATUS XML_StartCdataSectionWrite
    (
    XML_Output output    /* XML output object */
    )
```

Description

This routine begins a CDATA section. CDATA character data is written with **XML_CharacterDataWrite( )**. CDATA sections may not be nested and cause **XML_CharacterDataWrite( )** to generate output with no entitity expansion.

Returns

ERROR if a CDATA section is already active, **OK** otherwise.

Errno

S_xmlop_CDATA_ACTIVE

See also

xmlop, **XML_CharacterDataWrite( )**

Name

XML_EndCdataSectionWrite( ) - ends a CDATA section

Layer

EXPAT

Synopsis

```
STATUS XML_EndCdataSectionWrite
    (
    XML_Output output   /* XML output object */
    )
```

Description

This routine writes an end-of-CDATA marker. Outside the CDATA section, **XML_CharacterDataWrite( )** will return to doing entity expansion as necessary on character data.

Returns

ERROR if no CDATA section is active, **OK** otherwise.

Errno

S_xmlop_NO_CDATA_ACTIVE

See also

xmlop, **XML_StartCdataSectionWrite( )**

Name

XML_EntityRefWrite( ) - writes an entity reference

Layer

EXPAT

Synopsis

```
STATUS XML_EntityRefWrite
    (
    XML_Output     output,   /* XML output object */
    const XML_Char *name     /* entity name */
    )
```

Description

This routine writes an arbitrary entity reference to the output. Note that it is not necessary to use **XML_EntityRefWrite( )** for the predefined entities **&**, **<**, **>** as these are handled automatically by **XML_CharacterDataWrite( )**.

Returns

OK

Errno

N/A

See also

xmlop, **XML_CharacterDataWrite( )**

Name

XML_StartDoctypeDeclWrite( ) - starts a doctype declaration, does not print immediately if the Canonicalization option is turned on

Layer

EXPAT

Synopsis

```
STATUS XML_StartDoctypeDeclWrite
    (
    XML_Output     output,               /* XML output object */
    const XML_Char *doctypeName,         /* DOCTYPE name */
    const XML_Char *sysid,               /* SYSTEM attribute contents */
    const XML_Char *pubid,               /* PUBLIC attribute contents */
    int            has_internal_subset   /* not used, present for SAX completness */
    )
```

Description

This routine begins a DOCTYPE declaration. Either sysid, pubid or both may be **NULL**. If the Canonicalization option is turned on then this function does not print immediately but instead waits for the XML_EndDoctypeDeclWrite function to be called in order to sort any NOTATION writes.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_EndDoctypeDeclWrite( ) - ends a doctype declaration

Layer

EXPAT

Synopsis

```
STATUS XML_EndDoctypeDeclWrite
    (
    XML_Output output   /* XML output object */
    )
```

Description

This routine ends a DOCTYPE declaration. If the Canonicalization option is turned on then the entire DOCTYPE element is printed here, including any NOTATION declarations. Also all NOTATION declarations are sorted in lexicographic order.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_EntityDeclWrite( ) - writes a complete entity declaration

Layer

EXPAT

Synopsis

```
STATUS XML_EntityDeclWrite
    (
    XML_Output       output,                /* XML output object */
    const XML_Char * entityName,            /* entity name */
    int              is_parameter_entity,   /* indicates if this is a parameter entity */
    const XML_Char * value,                 /* entity value */
    int              value_length,          /* length of <value> */
    const XML_Char * base,                  /* not used in this routine */
    const XML_Char * systemId,              /* system ID */
    const XML_Char * publicId,              /* public ID */
    const XML_Char * notationName           /* notation name */
    )
```

Description

This routine is called for entity declarations. is_parameter_entity is non-zero if the entity is a parameter entity, zero otherwise.

For internal entities (<!ENTITY foo "bar">), value is non-**NULL** and systemId, publicID, and notationName is **NULL**. value is NOT **NULL**-terminated; the length is provided in value_length. Since it is legal to have zero-length values, do not use this argument to test for internal entities.

For external entities, value is **NULL** and systemId is non-**NULL**. publicId is **NULL** unless a public identifier was provided. notationName has a non-**NULL** value only for unparsed entity declarations.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_NotationDeclWrite( ) - write a complete notation declaration, this does not print immediately if the Canonicalization option is turned on

Layer

EXPAT

Synopsis

```
STATUS XML_NotationDeclWrite
    (
    XML_Output       output,          /* XML output object */
    const XML_Char * notationName,    /* notation name */
    const XML_Char * base,            /* base path */
```

```
    const XML_Char * systemId,       /* system ID */
    const XML_Char * publicId        /* public ID */
    )
```

Description

This routine writes a notation declaration to output. base is a base path set by the user, in the equivalent SAX API, this is set by XML_SetBase. The notationName will never be **NULL**; systemId and publicId may be **NULL**. If the Canonicalization option is turned on then the NOTATION declaration is stored and later sorted and printed when XML_EndDoctypeDeclWrite is called.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_AttlistDeclWrite( ) - writes a complete attlist declaration

Layer

EXPAT

Synopsis

```
STATUS XML_AttlistDeclWrite
    (
    XML_Output     output,      /* XML output object */
    const XML_Char *elname,
    const XML_Char *attname,
    const XML_Char *att_type,
    const XML_Char *dflt,
    int            isrequired
    )
```

Description

This routine is called for *each* attribute. A single Attlist declaration with multiple attributes will need to call this multiple times. The "default" parameter may be **NULL** in the case of the "#IMPLIED" or "#REQUIRED" keyword. The "isrequired" parameter is true and the default value is **NULL** in the case of "#REQUIRED". If "isrequired" is true and default is non-**NULL**, then this is a "#FIXED" default.

Returns

OK

Errno

N/A

See also

xmlop

Name

XML_ElementDeclWrite( ) - writes a complete element declaration

Layer

EXPAT

Synopsis

```
STATUS XML_ElementDeclWrite
    (
    XML_Output     output,   /* XML output object */
    const XML_Char *name,    /* ELEMENT name */
    XML_Content    *model    /* XML content model*/
    )
```

Description

This routine is called for an element declaration. See **XML_ParseXmlContent( )** for a description of the model argument. It is the caller's responsibility to free model when finished with it.

Returns

OK

Errno

N/A

See also

xmlop, **XML_ParseXmlContent( )**

4. XPP

Name

xpp - XML pull parser library

Layer

EXPAT

Routines

xppCreate( ) - create and initialize a new XML Pull Parser **xppAlloc( )** - allocate memory for the XML Pull Parser **xppInit( )** - initialize the XML Pull Parser **xppSetFileInput( )** - set the input file, and start parsing **xppSetStringInput( )** - set the input string, and start parsing **xppDestroy( )** - destroy an XML Pull Parser **xppSetExtEntBaseDir( )** - set the base directory for parsing external entities **xppSetExtEntParseEnable( )** - enable the external entity parsing feature **xppGetEventType( )** - get the current XPP event type **xppNext( )** - get the next XPP event type **xppGetName( )** - get the local name of the current start/end tag **xppGetNamespace( )** - get the namespace of the current start/end tag **xppGetQName( )** - get the namespace qualified name for the current start/end tag **xppLookupNamespace( )** - get the namespace for the specified prefix **xppGetNamespaceCount( )** - get the number of namespaces declared **xppGetPrefix( )** - get the prefix of the current start/end tag **xppGetAttributeCount( )** - get the number of attributes found in the current start tag **xppGetAttributeQName( )** - get the namespace qualified name of the attribute at the specified index **xppGetAttributeName( )** - get the local name of the attribute at the specified index **xppGetAttributeNamespace( )** - get the namespace of the attribute at the specified index **xppGetAttributePrefix( )** - get the prefix of the attribute at the specified index **xppGetAttributeValue( )** - get the value of the attribute at the specified index **xppGetAttributeValueByName( )** - get the value of the attribute for the specified namespace and name **xppGetTextLength( )** - gets the length of the text available **xppGetText( )** - get the currently available text **xppIsWhitespace( )** - determine whether the current TEXT node contains only whitespace **xppIsEmptyElementTag( )** - determine whether the current start tag is empty **xppGetColumnNumber( )** - get the current column position within the document being parsed **xppGetLineNumber( )** - get the current line position within the document being parsed **xppGetDepth( )** - get the current nesting depth **xppGetErrorString( )** - get the current XPP error string **xppGetFileName( )** - get the current XPP file name

Description

This library provides routines for making stream-based XML parsing easier by allowing the developer to iterate through an XML document rather than respond to events as is the typical interface provided by SAX based parsers.

It is very loosely based on the Java pull parser API described at http://www.xmlpull.org/v1/doc/api/org/xmlpull/v1/XmlPullParser.html

Here is an example of how to use the XML pull parser:

```
BOOL
xmlXppParseFile1
    (
    char *pBasePath,
    char *pFileName
    )
    {
    XPP_EVENT_T eventType;
    XPP *pXpp = NULL;
    FILE *parseFile;
    char buf[MAX_BUFFER_SIZE];

    strcpy(buf, pBasePath);
```

```
   strcat(buf, pFileName);

*******************************************
*    Initializing the pXpp data structure  *
*******************************************
   xppAlloc(&pXpp);
   xppInit(pXpp);

   printf("Initializing Pull Parser with file %s\n", buf);

   xppSetFileInput(pXpp, buf);
   eventType = xppGetEventType(pXpp);

*************************************************************************
*    Parse the XML document until the end of the document is reached    *
*************************************************************************
   while (eventType != END_DOC)
       {

************************************************
*    The start of the element is encountered    *
************************************************
       if (eventType == START_TAG)
           printf("found start tag: %s\n", xppGetQName(pXpp));

************************************************
*    The end of the element is encountered    *
************************************************
       else if (eventType == END_TAG)
           printf("found end tag: %s\n", xppGetQName(pXpp));

*************************************************************
*    The text of the element or attribute is encountered    *
*************************************************************
       else if (eventType == TEXT)
           {
           if (!xppIsWhitespace(pXpp))
               {
               XML_Char * text;
               int len = xppGetTextLength(pXpp);
               text = (XML_Char *)malloc(len * sizeof(XML_Char));
               if (text != NULL)
                   {
                   if (xppGetText(pXpp, text, len) == OK)
                       printf("found text: %s\n", text);
                   else
                       fprintf(stderr, "Error getting text: %s\n", xppGetErrorString(pXpp));
                   free (text);
                   }
               else
                   {
                   printf("found %d bytes of whitespace text\n", xppGetTextLength(pXpp));
                   }

               }
           }
```

```
        eventType = xppNext(pXpp);
        }

    *** End of Document Reached ***
    xppDestroy(pXpp);
    }
```

Include files

webservices/xml/xpp/xpp.h

Name

xppCreate( ) - create and initialize a new XML Pull Parser

Layer

EXPAT

Synopsis

```
XPP *xppCreate
    (
    void
    )
```

Description

This routine is a convenience wrapper around **xppAlloc( )** and **xppInit( )**, to provide greater consistency with the vxWorks model of **taskCreate( )**, **semBCreate( )**, etc.

Returns

pointer to the XPP data structure, or **NULL** if it could not be created

Errno

N/A

See also

xpp

Name

xppAlloc( ) - allocate memory for the XML Pull Parser

Layer

EXPAT

Synopsis

```
STATUS xppAlloc
    (
    XPP **pXpp   /* pointer to the XPP object to allocate for */
    )
```

Description

This routine allocates and zeroes memory for the XPP data structure.

Returns

OK if the memory was successfully allocated, **ERROR** otherwise.

Errno

N/A

See also

xpp

Name

xppInit( ) - initialize the XML Pull Parser

Layer

EXPAT

Synopsis

```
STATUS xppInit
    (
    XPP *pXpp   /* pointer to XPP object to initialize */
    )
```

Description

This routine initializes the XPP structure. It must be called before the parser is used.

Returns

OK if the structure is successfully initialized, **ERROR** otherwise.

Errno

N/A

See also

xpp

Name

xppSetFileInput( ) - set the input file, and start parsing

Layer

EXPAT

Synopsis

```
STATUS xppSetFileInput
    (
    XPP  *pXpp,      /* pointer to XPP object being used for parsing */
    char *fileName   /* name of file to parse */
    )
```

Description

This routine sets the input file for the XPP parser, and immediately starts a separate task to parse to the first event.

Returns

OK if the underlying SAX parser is started, **ERROR** otherwise.

Errno

N/A

See also

xpp

Name

xppSetStringInput( ) - set the input string, and start parsing

Layer

EXPAT

Synopsis

```
STATUS xppSetStringInput
    (
    XPP            *pXpp,       /* pointer to XPP object to parse with */
    const XML_Char *xmlString   /* pointer to string to parse */
    )
```

Description

This routine sets the input string for the XPP parser, and immediately starts a separate task to parse to the first event.

Returns

OK if the underlying SAX parser is started, **ERROR** otherwise.

Errno

N/A

See also

xpp

Name

xppDestroy( ) - destroy an XML Pull Parser

Layer

EXPAT

Synopsis

```
STATUS xppDestroy
    (
    XPP *pXpp   /* pointer to XPP object to recover resources from */
    )
```

Description

This routine kills the helper task, if necessary, closes any open files, and frees memory allocated by the XPP structure. The XPP pointer should not be used after this, as it will point to free memory on the heap.

Returns

OK

Errno

N/A

See also

xpp

Name

xppSetExtEntBaseDir( ) - set the base directory for parsing external entities

Layer

EXPAT

Synopsis

```
STATUS xppSetExtEntBaseDir
    (
    XPP  *pXpp,     /* pointer to XPP object to configure */
    char *baseDir   /* base path */
    )
```

Description

This routine sets the base directory for parsing external entities. This is usually the same directory as the file given in **xppSetFileInput( )**.

Returns

OK on success, **ERROR** on failure.

Errno

N/A

See also

xpp

Name

xppSetExtEntParseEnable( ) - enable the external entity parsing feature

Layer

EXPAT

Synopsis

```
void xppSetExtEntParseEnable
    (
    XPP *pXpp   /* pointer to XPP object to configure */
    )
```

Description

This routine enables the XML Pull Parser to parse external entities. It must be called before the parser is started with **xppSetFileInput( )** or **xppSetStringInput( )**.

Returns

N/A

Errno

N/A

See also

xpp

Name

xppGetEventType( ) - get the current XPP event type

Layer

EXPAT

Synopsis

```
XPP_EVENT_T xppGetEventType
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the current event type, which will be one of the following:

START_TAG

A starting element tag has been encountered, and all element data including its attributes are available (eg. <tag attr="value">) Note: This event type will also occur for empty tags, but will not be followed by an **END_TAG** event (eg. <tag attr="value"/>).

END_TAG

An ending element tag has been encountered, and all element data is available (eg. </tag>).

TEXT

Text contained within a tag has been encountered. (eg. <tag>text_found</tag>).

END_DOC

The end of the document has been reached.

DOC_ERROR

A serious error has occurred.

Returns

XPP_EVENT_T - the type of event

Errno

N/A

See also

xpp

Name

xppNext( ) - get the next XPP event type

Layer

EXPAT

Synopsis

```
XPP_EVENT_T xppNext
    (
    XPP *pXpp   /* pointer to XPP object */
    )
```

Description

This routine signals the underlying SAX parser to parse to the next event.

Returns

XPP_EVENT_T - the type of event (see **xppGetEventType( )**)

Errno

N/A

See also

xpp

Name

xppGetName( ) - get the local name of the current start/end tag

Layer

EXPAT

Synopsis

```
XML_Char *xppGetName
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the local (non-namespace-qualified) name of the current start/end tag.

Returns

name, or **NULL** if the current event is not **START_TAG** or **END_TAG**.

Errno

N/A

See also

xpp

Name

xppGetNamespace( ) - get the namespace of the current start/end tag

Layer

EXPAT

Synopsis

```
XML_Char *xppGetNamespace
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the namespace URI of the current start/end tag.

Returns

namespace, or **NULL** if the current event is not **START_TAG** or **END_TAG**.

Errno

N/A

See also

xpp

Name

xppGetQName( ) - get the namespace qualified name for the current start/end tag

Layer

EXPAT

Synopsis

```
XML_Char *xppGetQName
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the namespace qualified name for the current start/end tag.

Returns

name, or **NULL** if the current event is not **START_TAG** or **END_TAG**.

Errno

N/A

See also

xpp

Name

xppLookupNamespace( ) - get the namespace for the specified prefix

Layer

EXPAT

Synopsis

```
XML_Char *xppLookupNamespace
    (
    XPP      *pXpp,    /* pointer to XPP object to retrieve from */
    XML_Char *prefix   /* prefix to lookup namespace of */
    )
```

Description

This routine gets the namespace URI for the specified prefix.

Returns

namespace, or **NULL** if there is no namespace associated with the specified prefix.

Errno

N/A

See also

xpp

Name

xppGetNamespaceCount( ) - get the number of namespaces declared

Layer

EXPAT

Synopsis

```
int xppGetNamespaceCount
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the number of namespaces declared in the current document.

Returns

number of namespaces declared

Errno

N/A

See also

xpp

Name

xppGetPrefix( ) - get the prefix of the current start/end tag

Layer

EXPAT

Synopsis

```
XML_Char *xppGetPrefix
    (
```

```
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the prefix of the current start/end tag.

Returns

prefix, or **NULL** if the current event is not **START_TAG** or **END_TAG**.

Errno

N/A

See also

xpp

Name

xppGetAttributeCount( ) - get the number of attributes found in the current start tag

Layer

EXPAT

Synopsis

```
int xppGetAttributeCount
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the number of attributes available from the current start tag.

Returns

number of attributes, or -1 if the current event is not **START_TAG**.

Errno

N/A

See also

xpp

Name

xppGetAttributeQName( ) - get the namespace qualified name of the attribute at the specified index

Layer

EXPAT

Synopsis

```
XML_Char *xppGetAttributeQName
    (
    XPP *pXpp,    /* pointer to XPP object to retrieve from */
    int index     /* index of attribute name to retrieve */
    )
```

Description

This routine gets the namespace qualified name of the attribute at the specified index. index must be in the range: [0 ... **xppGetAttributeCount( )** - 1]

Returns

name, or **NULL** if the current event is not **START_TAG** or the index is out of range

Errno

N/A

See also

xpp

Name

xppGetAttributeName( ) - get the local name of the attribute at the specified index

Layer

EXPAT

Synopsis

```
XML_Char *xppGetAttributeName
    (
    XPP *pXpp,    /* pointer to XPP object to retrieve from */
    int index     /* index of attribute name to retrieve */
    )
```

Description

This routine gets the local (non-namespace-qualified) name of the attribute at the specified index. index must be in the range: [0 ... **xppGetAttributeCount( )** - 1]

Returns

name, or **NULL** if the current event is not **START_TAG** or the index is out of range.

Errno

N/A

See also

xpp

Name

xppGetAttributeNamespace( ) - get the namespace of the attribute at the specified index

Layer

EXPAT

Synopsis

```
XML_Char *xppGetAttributeNamespace
    (
    XPP *pXpp,    /* pointer to XPP object to retrieve from */
    int index     /* index of attribute to retrieve */
    )
```

Description

This routine gets the namespace URI of the attribute at the specified index. index must be in the range: [0 ... **xppGetAttributeCount( )** - 1]

Returns

namespace, or **NULL** if the current event is not **START_TAG** or the index is out of range.

Errno

N/A

See also

xpp

Name

xppGetAttributePrefix( ) - get the prefix of the attribute at the specified index

Layer

EXPAT

Synopsis

```
XML_Char *xppGetAttributePrefix
    (
    XPP *pXpp,    /* pointer to XPP object to retrieve from */
    int index     /* index of attribute to retrieve */
    )
```

Description

This routine gets the prefix of the attribute at the specified index. index must be in the range: [0 ... **xppGetAttributeCount( )** -1]

Returns

prefix, or **NULL** if the current event is not **START_TAG** or the index is out of range.

Errno

N/A

See also

xpp

Name

xppGetAttributeValue( ) - get the value of the attribute at the specified index

Layer

EXPAT

Synopsis

```
XML_Char *xppGetAttributeValue
    (
    XPP *pXpp,    /* pointer to XPP object to retrieve from */
    int index     /* index of attribute value to retrieve */
    )
```

Description

This routine gets the value of the attribute at the specified index. index must be in the range: [0 ... **xppGetAttributeCount( )** - 1]

Returns

value string, or **NULL** if the current event is not **START_TAG** or the index is out of range.

Errno

N/A

See also

xpp

Name

xppGetAttributeValueByName( ) - get the value of the attribute for the specified namespace and name

Layer

EXPAT

Synopsis

```
XML_Char *xppGetAttributeValueByName
    (
    XPP      *pXpp,        /* pointer to XPP object to retrieve from */
    XML_Char *nameSpace,   /* namespace of the attribute to retrieve */
    XML_Char *name         /* name of attribute to retrieve */
    )
```

Description

This routine gets the value of the attribute for the specified namespace and name.

Returns

value string, or **NULL** if the attribute is not found or an error occurs.

Errno

N/A

See also

xpp

Name

xppGetTextLength( ) - gets the length of the text available

Layer

EXPAT

Synopsis

```
int xppGetTextLength
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the length of the text available.

This is typically called before calling **xppGetText( )** to allocate a buffer large enough to store the text. The value returned includes an extra byte for the terminating **/0**.

Returns

The number of bytes of text available, or -1 if the current event is not TEXT.

Errno

N/A

See also

xpp

Name

xppGetText( ) - get the currently available text

Layer

EXPAT

Synopsis

```
STATUS xppGetText
    (
    XPP      *pXpp,      /* pointer to XPP object to retrieve from */
    XML_Char *pBuffer,   /* buffer to return text into */
    int      len         /* length of pBuffer */
    )
```

Description

This routine copies the currently available text into the buffer. Memory must already be allocated to the buffer. The number of bytes that will be written to the buffer is **xppGetTextLength( )** + 1.

Returns

OK on success, **ERROR** otherwise.

Errno

N/A

See also

xpp

Name

xppIsWhitespace( ) - determine whether the current TEXT node contains only whitespace

Layer

EXPAT

Synopsis

```
BOOL xppIsWhitespace
    (
    XPP *pXpp   /* pointer to XPP object to analyze */
    )
```

Description

This routine indicates whether the current TEXT node contains only whitespace.

Returns

TRUE if the current TEXT node contains only whitespace or the current event type is not TEXT. Otherwise **FALSE** is returned.

Errno

N/A

See also

xpp

Name

xppIsEmptyElementTag( ) - determine whether the current start tag is empty

Layer

EXPAT

Synopsis

```
BOOL xppIsEmptyElementTag
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine indicates whether the current start tag is empty or not. An empty tag looks like: <tag/>

Returns

TRUE if the current start tag is empty, **FALSE** otherwise.

Errno

N/A

See also

xpp

Name

xppGetColumnNumber( ) - get the current column position within the document being parsed

Layer

EXPAT

Synopsis

```
int xppGetColumnNumber
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the current column position of the underlying SAX parser.

Returns

current column position

Errno

N/A

See also

xpp

Name

xppGetLineNumber( ) - get the current line position within the document being parsed

Layer

EXPAT

Synopsis

```
int xppGetLineNumber
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the current line number of the underlying SAX parser.

Returns

current line number

Errno

N/A

See also

xpp

Name

xppGetDepth( ) - get the current nesting depth

Layer

EXPAT

Synopsis

```
int xppGetDepth
    (
    XPP *pXpp   /* pointer to XPP object to retrieve from */
    )
```

Description

This routine gets the current nesting depth. The nesting depth of the first element is 0 and is increased by one for each level.

Returns

current nesting depth

Errno

N/A

See also

xpp

Name

xppGetErrorString( ) - get the current XPP error string

Layer

EXPAT

Synopsis

```
char *xppGetErrorString
    (
    XPP *pXpp
    )
```

Description

This routine gets the current error string. This is only valid after an error has been reported with a **DOC_ERROR** event.

Returns

pointer to error string

Errno

N/A

See also

xpp

Name

xppGetFileName( ) - get the current XPP file name

Layer

EXPAT

Synopsis

```
char *xppGetFileName
    (
    XPP *pXpp
    )
```

Description

This routine gets the current parse file name. This could be different from the name given to **xppSetFileInput( )**, if the parser has opened an external entity.

Returns

file name string

Errno

N/A

See also

xpp